

WHY DOES HEAP EXIST?

Imagine you run an **emergency room**. Patients arrive constantly. You don't treat them in arrival order — you treat the **most critical patient first**.

A sorted array would be too slow to re-sort every time a new patient arrives. A Heap lets you always find the "most important" in **O(1)** and insert/remove in **O(log n)**.

AHA MOMENT

When you need the **extreme element** (min or max) repeatedly and efficiently — that's a Heap. Not sorting. Sorting gives you everything ordered. Heap gives you the *one thing that matters right now*.

1 WHAT IS A HEAP?

- A **Complete Binary Tree** stored as an **array**
- Satisfies the **Heap Property** (parent vs children order)
- **NOT** sorted — only root is guaranteed extreme
- Implements a **Priority Queue** ADT

MIN HEAP
— Parent
≤ Children

MAX HEAP — Parent ≥ Children

```
graph TD
    classDef r fill:#fff1f2,stroke:#dc2626,stroke-width:2px,color:#991b1b,font-weight:700;
    classDef b fill:#eff6ff,stroke:#2563eb,stroke-width:2px,color:#1e40af,font-weight:700;
    A((1)) ---> B((3))
    A ---> C((2))
    B ---> D((7))
    B ---> E((5))
    C ---> F((4))
    C ---> G((6))
```

2 ARRAY REPRESENTATION (0-indexed)

```
graph TD
    classDef b fill:#eff6ff,stroke:#2563eb,stroke-width:2px,color:#1e40af,font-weight:700;
    A((1)) ---> B((3))
    A ---> C((2))
    B ---> D((7))
    B ---> E((5))
    C ---> F((4))
    C ---> G((6))
```

Index	0	1	2	3	4	5	6
Value	1	3	2	7	5	4	6

```
parent(i) = (i-1)/2
left(i) = 2*i + 1
right(i) = 2*i + 2
```

PREREQUISITES TO KNOW FIRST

Concept	Why needed
Complete Binary Tree	Heap is implemented as one
Tree height = O(log n)	Drives all heap complexities
Array indexing	Left/Right/Parent formulas
Comparator (Java)	Custom ordering in PQ
Bubble Up / Sift Down	Core heap operations
Priority Queue ADT	What heap implements

3 TIME COMPLEXITIES

Operation	Time	Why
offer(x) / insert	O(log n)	Bubble Up (height = log n)
poll() / remove root	O(log n)	Sift Down (height = log n)
peek()	O(1)	Root is always index 0
size() / isEmpty()	O(1)	Counter field
Build Heap (n items)	O(n)	Bottom-up heapify (not n log n!)
contains(x)	O(n)	No index — must scan

★ Build Heap = O(n) — better than n × offer() = O(n log n)

HEAPIFY — HOW IT WORKS

Insert → Bubble Up (Heapify Up)

1. Add element at last position
2. Compare with parent
3. If violates heap property → swap
4. Repeat until root or satisfied

Delete Root → Sift Down (Heapify Down)

1. Swap root with last element
2. Remove last (was root)
3. Compare root with children → swap with smaller (min) or larger (max)
4. Repeat until leaf or satisfied

4 JAVA PRIORITYQUEUE — COMPLETE API

Min Heap (Default)

```
// Smallest at peek()
PriorityQueue<Integer> m
inH
= new PriorityQueue<>
();
```

Max Heap (3 Ways)

```
// 1. Recommended
PriorityQueue<Integer> m
axH =
    new PriorityQueue<>(
        Collections.reverseO
rder());

// 2. Lambda
PriorityQueue<Integer> m
axH =
    new PriorityQueue<>(
        (a, b) -> b - a);

// 3. Integer.compare (s
afe)
new PriorityQueue<>(
    (a,b) -> Integer.compa
re(b,a));
```

Common Operations

Method	What it does
offer(x)	Insert element — use this always
poll()	Remove & return root (null if empty)
peek()	View root without removing (null if empty)
size()	Number of elements — O(1)
isEmpty()	Empty check — O(1)
contains(x)	Search — O(n) ⚠️ avoid in hot loops

✗ **b - a** comparator can **overflow** for large integers. Always prefer `Integer.compare(b, a)`.

5 CUSTOM COMPARATORS — ALL PATTERNS

Pair / Object Heaps

```
// Min Heap by first element of int[]
PriorityQueue<int[]> pq =
    new PriorityQueue<>((a, b) -> a[0] - b[0]);

// Max Heap by first element
new PriorityQueue<>((a, b) -> b[0] - a[0]);

// Sort by distance (double) — K Closest Points
PriorityQueue<int[]> pq = new PriorityQueue<>(
    (a, b) -> Double.compare(
        a[0]*a[0]+a[1]*a[1], b[0]*b[0]+b[1]*b[1]));

// Multi-level sort: freq desc, then alpha asc
PriorityQueue<Map.Entry<String,Integer>> pq =
    new PriorityQueue<>((a, b) ->
        a.getValue() != b.getValue() ?
        b.getValue() - a.getValue() :
        a.getKey().compareTo(b.getKey()));

// Heap of custom class
class Task { int time, cpu; }
PriorityQueue<Task> pq = new PriorityQueue<>(
    Comparator.comparingInt(t -> t.time));
```

💡 For String keys → use `.compareTo()`, never subtraction

HEAPUTILS — REUSABLE TOOLKIT (COPY-PASTE READY)

Build Heap from Array O(n)

```
// Java: pass collection to constru
ctor
int[] arr = {5, 3, 1, 7, 2};
PriorityQueue<Integer> pq =
    new PriorityQueue<>();
for (int x : arr) pq.offer(x);

// Or from List — O(n) build
```

Heap Sort (ascending)

```
int[] heapSort(int[] arr) {
    PriorityQueue<Integer> pq =
        new PriorityQueue<>();
    for (int x : arr) pq.offer(x);
    int[] res = new int[arr.length];
    for (int i = 0; !pq.isEmpty(); i++)
        res[i] = pq.poll();
}
```

Kth Largest Utility

```
int kthLargest(int[] nums, int k) {
    // Min Heap of size k
    PriorityQueue<Integer> pq =
        new PriorityQueue<>();
    for (int x : nums) {
        pq.offer(x);
        if (pq.size() > k) pq.poll();
    }
}
```

PATTERN 1 REPEATED MAX / MIN RETRIEVAL e.g. Last Stone Weight, Task Scheduler

WHEN TO USE - Always pick the largest (or smallest) element - Do an operation on it - Put the result back - Repeat until 1 or 0 left AHA The heap keeps the "king" at the top automatically after every operation.	HEAP TO USE Max Heap <pre>graph TD classDef r fill:#fff1f2,stroke:#dc2626,stroke-width:2px,color:#991b1b,font-weight:700; A((9))--r-->B((6))--r-->C((7))--r-->D((5))</pre> Largest always at top	TEMPLATE (JAVA) <pre>// Last Stone Weight pattern PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.re verseOrder()); for (int s : stones) pq.offer(s); while (pq.size() > 1) { int a = pq.poll(); // largest int b = pq.poll(); // 2nd largest if (a != b) pq.offer(a - b); } return pq.isEmpty() ? 0 : pq.peak();</pre>	COMPLEXITY Time: $O(n \log n)$ (n polls, each $O(\log n)$) Space: $O(n)$ LC 1046 Last Stone Weight LC 621 Task Scheduler
--	--	---	---

PATTERN 2 TOP-K ELEMENTS e.g. Kth Largest, Top K Frequent, K Closest Points

WHEN TO USE - Need K largest elements → use Min Heap size K - Need K smallest elements → use Max Heap size K - Need K most frequent → freq as key - Key trick: <i>discard the worst, keep the best K</i> AHA "Keep the K best" = use a heap of <i>opposite type</i> as a filter. Min Heap removes smallest → keeps K largest.	COUNTERINTUITIVE RULE Top K Largest → Min Heap (size K) Top K Smallest → Max Heap (size K) Root = the "weakest survivor" that gets replaced if better comes	TEMPLATE — TOP K LARGEST & SMALLEST <pre>// Top K LARGEST → Min Heap of size K PriorityQueue<Integer> pq = new PriorityQ ueue<>(); for (int x : nums) { pq.offer(x); if (pq.size() > k) pq.poll(); // evict smallest } // pq now has the K largest // Top K SMALLEST → Max Heap of size K PriorityQueue<Integer> pq2 = new PriorityQueue<>(Collections.reverse Order()); for (int x : nums) { pq2.offer(x); if (pq2.size() > k) pq2.poll(); // evic t largest }</pre>	COMPLEXITY Time: $O(n \log k)$ (heap size stays K) Space: $O(k)$ vs Sorting: $O(n \log n) + O(n)$ LC 215 Kth Largest LC 347 Top K Frequent LC 973 K Closest Points
--	--	--	---

PATTERN 3 MERGE K SORTED LISTS / ARRAYS e.g. Merge K Sorted Lists, Find K Pairs Smallest Sums

WHEN TO USE

- Have K sorted sources (lists, arrays, rows)
- Need to merge into one sorted stream
- Always pick the globally smallest current candidate
- Pattern: poll() one → push its next

AHA

Heap acts as a "tournament bracket" — K competitors, always the current winner advances.

VISUAL

graph LR A1((1))-->A2((4))-->A3((7))
B1((2))-->B2((5))-->B3((8)) C1((3))-->C2((6))-->C3((9))
A1--H[MinHeap] B1--H C1--H
Only K items in heap at once

TEMPLATE — MERGE K SORTED LISTS

```
// Store [value, listIndex, nodeIndex]
PriorityQueue<int[]> pq =
    new PriorityQueue<((a, b) -> a[0] - b[0]);

// Init: add head of each list
for (int i = 0; i < lists.length; i++)
    if (lists[i] != null)
        pq.offer(new int[]{lists[i].val, i, 0});

ListNode dummy = new ListNode(0), tail = dummy;
while (!pq.isEmpty()) {
    int[] cur = pq.poll();
    tail.next = lists[cur[1]]; tail = tail.next;
    lists[cur[1]] = lists[cur[1]].next;
    if (lists[cur[1]] != null)
        pq.offer(new int[]{lists[cur[1]].val, cur[1], 0});
}
return dummy.next;
```

COMPLEXITY

- **Time:** O(N log k)
N=total nodes, k=lists
 - **Space:** O(k)
heap has at most k items
- LC 23 Merge K Sorted Lists

LC 373 Find K Pairs Smallest Sums

LC 378 Kth Smallest in Matrix

PATTERN 4 TWO HEAPS — RUNNING MEDIAN e.g. Find Median from Data Stream, Sliding Window Median

WHEN TO USE

- Stream of numbers arrives one by one
- Need **median at any point**
- O(log n) insert, O(1) median query
- Split stream into two halves

AHA

Max Heap (left half) top = lower bound.
Min Heap (right half) top = upper bound.

VISUAL — TWO HEAPS

Max Heap (left / smaller half) graph TD classDef r fill:#fff1f2,stroke:#dc2626,stroke-width:2px,color:#991b1b,font-weight:700; A((5)):::r-->B((2)):::r A-->C((4)):::r

Min Heap (right / larger half) graph TD classDef g fill:#f0fdf4,stroke:#059669,stroke-width:2px,color:#065f46,font-weight:700; D((7)):::g-->E((9)):::g D-->F((10)):::g

Median = (5+7)/2 = 6

TEMPLATE — ADDNUM + FINDMEDIAN

```
PriorityQueue<Integer> maxH =
    new PriorityQueue<>(Collections.reverseOrder());
PriorityQueue<Integer> minH =
    new PriorityQueue<>();

void addNum(int num) {
    maxH.offer(num);
    minH.offer(maxH.poll());
    // balance
    if (maxH.size() < minH.size())
        maxH.offer(minH.poll());
}

double findMedian() {
```

COMPLEXITY

- **addNum:** O(log n)
 - **findMedian:** O(1)
 - **Space:** O(n)
- LC 295 Find Median from Data Stream

LC 480 Sliding Window Median

PATTERN 5 GREEDY SCHEDULING / COOLDOWN e.g. Task Scheduler, Reorganize String, Process Tasks Using Servers

WHEN TO USE

- Tasks have **cooldown** before repeating
- Always process **most frequent** first
- Use Max Heap for frequency
- Use Queue to hold "cooling down" items

⚡ AHA

Greedy: always pick the busiest available worker. Use a queue as a "waiting room" during cooldown.

TEMPLATE — TASK SCHEDULER (LC 621)

```
Map<Character, Integer> freq = new HashMap<>();
for (char c : tasks) freq.merge(c, 1, Integer::sum);

PriorityQueue<Integer> maxH =
    new PriorityQueue<>(Collections.reverseOrder());
maxH.addAll(freq.values());

Queue<int[]> q = new LinkedList<>(); // [count, readyTime]
int time = 0;

while (!maxH.isEmpty() || !q.isEmpty()) {
    time++;
    if (!maxH.isEmpty()) {
        int cnt = maxH.poll() - 1;
        if (cnt > 0) q.offer(new int[]{cnt, time + cnt});
    }
    if (!q.isEmpty() && q.peek()[1] == time)
        maxH.offer(q.poll()[0]);
}
return time;
```

COMPLEXITY

- **Time:** O(n log n)
- **Space:** O(n)

- LC 621 Task Scheduler
- LC 767 Reorganize String
- LC 1405 Longest Happy String
- LC 1882 Process Tasks Using Servers

PATTERN 6 INTERVAL TRACKING & RESOURCE ALLOCATION

e.g. Meeting Rooms II, Single-Threaded CPU, Seat Reservation

WHEN TO USE

- Intervals/events processed chronologically
- Track currently **active resources** (rooms, CPU)
- Min Heap holds **end times** of active items
- If smallest end time ≤ next start time → reuse resource

⚡ AHA

Sort intervals by start time. Min Heap tracks end times. The root of the heap represents the earliest resource to become free.

TEMPLATE — MEETING ROOMS II (LC 253)

```
int minMeetingRooms(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    // Min Heap stores end times of active meetings
    PriorityQueue<Integer> heap = new PriorityQueue<>();

    for (int[] interval : intervals) {
        // Free room if earliest ending meeting is done
        if (!heap.isEmpty() && heap.peek() <= interval[0]) {
            heap.poll();
        }
        heap.offer(interval[1]); // allocate room
    }
    return heap.size(); // min rooms needed
}
```

COMPLEXITY

- **Time:** O(n log n)
- **Space:** O(n)

- LC 253 Meeting Rooms II
- LC 1834 Single-Threaded CPU
- LC 1845 Seat Reservation Manager
- LC 2402 Meeting Rooms III

HEAP — WHICH DATA STRUCTURE TO USE?

Start: What does the problem need?

Repeatedly pull the LARGEST element

→

Max Heap
Collection

Repeatedly pull the SMALLEST element

→

Min Heap
new PriorityQu()

Keep Top K LARGEST elements

→

Min Heap, smallest size K
evict when size > K

Keep Top K SMALLEST elements

→

Max Heap, largest size K
evict when size > K

Merge K sorted lists/streams

→

Min Heap
1 candidate from each source

Running MEDIAN of a stream

→

Two Heaps
Max (left) + Min (right)

Active intervals / room allocation

→

Min Heap (end times)
sort by start time, heap for end time

Tasks with cooldown / scheduling

→

Max Heap + Queue
freq-based, cooldown, waiting room

💡 The Core Question: What element should come out next? Put that on top.

PATTERN RECOGNITION — TRIGGER WORDS

Trigger Word(s)	Pattern	Heap Type
K largest, Kth largest, top K	Top-K	Min Heap K
K smallest, Kth smallest, closest K	Top-K	Max Heap K
K most frequent, top K frequent	Top-K + freq map	Min Heap K
merge K, K sorted lists/arrays	Merge K	Min Heap
median, data stream, sliding median	Two Heaps	Max+Min
last stone, always pick two largest	Repeated Max	Max Heap
meeting rooms, active intervals, CPU tasks	Interval Track	Min Heap (end times)
cooldown, scheduler, reorganize	Greedy Sched.	Max Heap + Queue
process in order of priority/weight	Simulation	Custom PQ

INTERVIEW FOLLOW-UPS — BE READY

Interviewer Asks	Your Answer
What if there are duplicates?	Heap handles them naturally. Use stable comparison if order matters.
Can you do it in O(n) instead of O(n log n)?	QuickSelect for Kth element. But heap is simpler & more general.
What's build heap O(n) vs insert n times O(n log n)?	Bottom-up heapify skips half the nodes (leaves don't sift). Amortized O(n).
Why not just sort?	Heap = O(n log k) vs Sort = O(n log n). Much faster when k << n.
Unsorted intervals in Meeting Rooms II?	Must sort by start time FIRST. Min Heap only manages active end times.
Two heaps space complexity?	O(n) — stores all elements across both heaps.

EASY — Core Intuition

LC 703 · Kth Largest Element in StreamEASY

Pattern: Top-K (Min Heap size K)

Key Insight: Min heap size K. Root = Kth largest.

Min HeapStream

```
class KthLargest {
    PriorityQueue<Integer> pq = new Priority
    Queue<>(); int k;
    KthLargest(int k, int[] nums) { this.k
    = k; for (int x : nums) add(x); }
    int add(int val) {
        pq.offer(val); if (pq.size() > k) p
        q.poll();
        return pq.peek();
    }
}
```

LC 1046 · Last Stone WeightEASY

Pattern: Repeated Max Retrieval

Key Insight: Max Heap — smash 2 heaviest stones.

Max Heap

```
int lastStoneWeight(int[] stones) {
    PriorityQueue<Integer> pq = new Priority
    Queue<>(Collections.reverseOrder());
    for (int s : stones) pq.offer(s);
    while (pq.size() > 1) {
        int a = pq.poll(), b = pq.poll();
        if (a != b) pq.offer(a - b);
    }
    return pq.isEmpty() ? 0 : pq.peek();
}
```

LC 215 · Kth Largest Element in ArrayMEDIUM

Pattern: Top-K

Key Insight: Min Heap of size K. Peek = Kth largest.

Min HeapQuickSelect

```
int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> pq = new Priority
    Queue<>();
    for (int x : nums) { pq.offer(x); if
    (pq.size() > k) pq.poll(); }
    return pq.peek();
}
```

MEDIUM — Top-K & Scheduling

LC 973 · K Closest Points to OriginMEDIUM

Pattern: Top-K Smallest

Key Insight: Max Heap size K by distance.

Max Heap

```
int[][] kClosest(int[][] pts, int k) {
    PriorityQueue<int[]> pq = new Priority
    Queue<>();
    (a,b)→(b[0]*b[0]+b[1]*b[1])-(a[0]*a
    [0]+a[1]*a[1]);
    for (int[] p : pts) { pq.offer(p); if
    (pq.size() > k) pq.poll(); }
    return pq.toArray(new int[k][]);
}
```

LC 347 · Top K Frequent ElementsMEDIUM

Pattern: Top-K Frequency

Key Insight: HashMap count → Min Heap size K.

Min HeapMap

```
int[] topKFrequent(int[] nums, int k) {
    Map<Integer,Integer> f = new HashMap<>
    ();
    for (int x : nums) f.merge(x, 1, Integ
    er::sum);
    PriorityQueue<Map.Entry<Integer,Integer>> pq =
    new PriorityQueue<>(Comparator.compa
    ringInt(Map.Entry::getValue));
    for (var e : f.entrySet()) { pq.offer
    (e); if (pq.size() > k) pq.poll(); }
    return pq.stream().mapToInt(Map.Entr
    y::getKey).toArray();
}
```

LC 621 · Task SchedulerMEDIUM

Pattern: Cooldown Sched.

Key Insight: Max Heap freq. Process busiest task.

Max Heap

```
int leastInterval(char[] tasks, int n) {
    int[] freq = new int[26];
    for (char c : tasks) freq[c-'A']++;
    int maxF = 0, maxC = 0;
    for (int f : freq) maxF = Math.max(max
    F, f);
    for (int f : freq) if (f == maxF) maxC
    ++;
    return Math.max(tasks.length, (maxF-1)
    *(n+1)+maxC);
}
```

MEDIUM — Interval Allocation

LC 253 · Meeting Rooms II

MEDIUM

Pattern: Interval Track
Key Insight: Sort start time, Min Heap end times.

Min Heap

```
int minMeetingRooms(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    PriorityQueue<Integer> heap = new PriorityQueue<>();
    for (int[] it : intervals) {
        if (!heap.isEmpty() && heap.peek() <= it[0]) heap.poll();
        heap.offer(it[1]);
    }
    return heap.size();
}
```

LC 23 · Merge K Sorted Lists

HARD

Pattern: Merge K Sorted
Key Insight: Min Heap head nodes. Poll → push next.

Min Heap

```
ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> pq = new PriorityQueue<>((a,b) -> a.val - b.val);
    for (ListNode h : lists) if (h != null) pq.offer(h);
    ListNode d = new ListNode(0), t = d;
    while (!pq.isEmpty()) {
        t.next = pq.poll(); t = t.next;
        if (t.next != null) pq.offer(t.next);
    }
    return d.next;
}
```

LC 295 · Find Median Data Stream

HARD

Pattern: Two Heaps
Key Insight: Max Heap (left) + Min Heap (right).

Two Heaps

```
class MedianFinder {
    PriorityQueue<Integer> lo = new PriorityQueue<>(Collections.reverseOrder());
    PriorityQueue<Integer> hi = new PriorityQueue<>();
    void addNum(int num) {
        lo.offer(num); hi.offer(lo.poll());
        if (lo.size() < hi.size()) lo.offer(hi.poll());
    }
    double findMedian() {
        return lo.size() > hi.size() ? lo.peek() : (lo.peek() + hi.peek()) / 2.0;
    }
}
```

HARD & EXPERT — Advanced Heap

LC 1851 · Min Interval for Each Query

HARD

Pattern: Sweep Line + Min Heap
Key Insight: Sort queries & intervals. Push active, pop expired.

Min Heap

```
int[] minInterval(int[][] ivals, int[] queries) {
    int n=queries.length; int[] idx=new int[n],res=new int[n];
    for(int i=0;i<n;i++) idx[i]=i;
    Arrays.sort(idx,(a,b)->queries[a]-queries[b]);
    Arrays.sort(ivals,(a,b)->a[0]-b[0]);
    PriorityQueue<int[]> pq=new PriorityQueue<>((a,b)->a[0]-b[0]);
    int i=0; Arrays.fill(res,-1);
    for(int qi:idx){int q=queries[qi];
        while(i<ivals.length&&ivals[i][0]<=q){
            pq.offer(new int[]{ivals[i][1]-ivals[i][0]+1,ivals[i][1]});i++;
        }
        while(!pq.isEmpty()&&pq.peek()[1]<q)pq.poll();
        if(!pq.isEmpty())res[qi]=pq.peek()[0];}return res;
}
```

LC 632 · Smallest Range K Lists

HARD

Pattern: Merge K + Min Heap
Key Insight: Track global max, shrink range by advancing min.

Min Heap

```
// Min Heap stores [val, listIdx, elemIdx]
// Track global max. Range = [heap_min, global_max]
// Advance pointer of min element list until exhausted
```

COMPLETE PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Diff
703	Kth Largest Stream	Top-K	E
1046	Last Stone Weight	Repeat Max	E
215	Kth Largest Array	Top-K	M
973	K Closest Points	Top-K Min	M
347	Top K Frequent	Top-K Freq	M
621	Task Scheduler	Scheduling	M
253	Meeting Rooms II	Interval Track	M
23	Merge K Lists	Merge K	H
295	Find Median Stream	Two Heaps	H
1851	Min Interval Query	Sweep+Heap	H

🔍 DRY RUN — Build Heap + Poll

Insert: 5, 3, 8, 1, 4 into Min Heap

Step	Action	Array State	Heap State
1	Insert 5	[5]	Root=5 ✓
2	Insert 3 → Bubble Up	[5,3] → [3,5]	3<5 → Swap
3	Insert 8	[3,5,8]	8>3 ✓ No swap
4	Insert 1 → Bubble Up	[3,5,8,1] → [3,1,8,5] → [1,3,8,5]	1<5→swap, 1<3→swap
5	Insert 4 → Bubble Up	[1,3,8,5,4] → [1,3,8,5,4]	4>3 ✓

Poll (remove min = 1):

Step	Action	Array State
1	Swap root(1) with last(4)	[4,3,8,5,1]
2	Remove last (was root=1)	[4,3,8,5]
3	Sift Down 4: children 3,8 → 3 is min → Swap	[3,4,8,5]
4	Sift Down 4: children 5 → 5>4 ✓ Stop	[3,4,8,5] ✓

Each bubble up/sift down traverses at most **log₂(n) levels = height of tree**

📐 COMPLEXITY DERIVATION (Not Memorized — Derived!)

Why is poll/offer O(log n)?

Complete binary tree with n nodes has height = **⌊log₂n⌋**
Bubble Up / Sift Down does at most 1 comparison per level.
Therefore: **T = O(height) = O(log n)**

Why is Build Heap O(n) not O(n log n)?

Only nodes at internal levels need sift-down. Leaves (n/2 nodes) do 0 work.
Total work = Σ (nodes at height h) × h
= n/4 × 1 + n/8 × 2 + n/16 × 3 + ...
= n × Σ(h/2^h) = n × 2 = **O(n)**

This geometric series converges to 2, proving O(n) build heap.

Why Top-K is O(n log k) not O(n log n)?

Heap stays at most size K throughout.
Each offer + possible poll = O(log K).
n elements processed = **n × O(log k) = O(n log k)**
When k = n, this = O(n log n). When k << n, huge speedup!

❌ COMMON MISTAKES

- Using `b-a` comparator → **integer overflow!** Use `Integer.compare(b,a)`
- Calling `poll()` on empty heap → returns null, causes NPE
- Thinking default Java PQ is Max Heap → it's **Min Heap**
- Assuming heap gives sorted order → **it doesn't**
- Using `contains()` for existence check → **O(n)**, not O(log n)
- Forgetting to sort intervals by start time before using heap for end times
- Top-K confusion: Top K Largest → Min

🔗 EDGE CASES — ALWAYS CHECK

- Empty heap:** `peek()` returns null, `poll()` returns null
- K = n:** Top-K returns entire array
- K = 1:** Min Heap of size 1 = running minimum
- All same elements:** Two heaps — equal split
- Single element:** Median = that element itself
- Interval overlap edge case:** Meeting ends at t=5 and next starts at t=5 (<= vs <)
- Extreme values:** Summing priorities

🔗 HEAP vs ALTERNATIVES

Operation	Heap	Sorted Array	TreeSet
Insert	O(log n)	O(n)	O(log n)
Get Min/Max	O(1)	O(1)	O(log n)
Delete Root	O(log n)	O(n)	O(log n)
Arbitrary Delete	O(n)	O(n)	O(log n)
Search	O(n)	O(log n)	O(log n)

DECLARE

```
// Min Heap
PQ<Integer> mn = new PQ<>();
// Max Heap
PQ<Integer> mx = new PQ<>(
    Collections.reverseOrder
());
// Custom
PQ<int[]> pq = new PQ<>(
    (a,b)→a[0]-b[0]);
```

INSERT/REMOVE

```
pq.offer(x); // insert
pq.poll();  // remove root
pq.peek();  // view root
pq.size();  // count
pq.isEmpty(); // check
// Add + cap at K:
pq.offer(x);
if(pq.size()>k) pq.poll();
```

PATTERN MAP

```
Max repeating → MaxH
Top K Largest → MinH(K)
Top K Smallest → MaxH(K)
Merge K sorted → MinH
Running Median → 2 Heaps
Interval Track → MinH(end)
```

KEY COMPLEXITIES

```
offer / poll → O(log n)
peek → O(1)
Build Heap → O(n)
Top-K → O(n log k)
Merge K (N tot) → O(N log k)
Sweep+Heap → O(n log n)
```

⚡ COMPLETE PATTERN REFERENCE — KNOW THIS COLD

Pattern	Trigger Keywords	Heap Type	Size Constraint	Key Operation	Complexity
Repeated Max/Min	always pick largest, smash, eliminate	Max or Min Heap	Unbounded	poll → operate → offer(result)	O(n log n)
Top K Largest	Kth largest, top K, K biggest	Min Heap	Size = K	offer + if size>K poll	O(n log k)
Top K Smallest	Kth smallest, K closest, K lightest	Max Heap	Size = K	offer + if size>K poll	O(n log k)
Merge K Sorted	merge K lists, K sorted streams	Min Heap	Size = K	poll node → offer next	O(N log k)
Two Heaps	median, data stream, running median	Max + Min	diff ≤ 1	addNum rebalance, findMedian O(1)	O(n log n)
Interval Track	meeting rooms, active intervals, CPU	Min Heap	Unbounded	sort start, min heap end time	O(n log n)
Greedy Sched.	cooldown, task, reorganize, server	Max Heap + Queue	Freq based	always pick busiest available	O(n log n)

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Explain Min vs Max Heap without looking

☐ Write Min Heap and Max Heap in Java (3 ways)

☐ Trace insert + sift-up on paper

☐ Trace poll + sift-down on paper

☐ Derive O(n) build heap (geometric series)

☐ Explain why Top-K Largest uses Min Heap

☐ Code Top-K without looking

☐ Code Merge K Sorted Lists from memory

☐ Code Two Heaps MedianFinder

☐ Code Meeting Rooms II with Min Heap

☐ Identify heap pattern from problem keywords

☐ Write custom comparator for int[], pairs, objects

☐ Solve LC 215, 347, 23, 295 within 20 min each

☐ Explain lazy deletion for

🏠 GOLDEN RULES — NEVER FORGET

Rule 1 — The Opposition Rule

```
To keep the K largest → use a Min Heap (evict the smallest intruder)
To keep the K smallest → use a Max Heap (evict the largest intruder)
```

Rule 2 — The Heap is NOT sorted

```
Heap only guarantees root is min/max. The rest is unordered. Never iterate a heap expecting order.
```

Rule 3 — The Overflow Trap